Szekvenciális fájl feldolgozása

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

Szekvenciális inputfájl

- A szekvenciális inputfájl (infile(E)) elnevezést olyan gyűjteményre használjuk, amelynek elemeit (amelyek mind egy adott E halmazhoz tartoznak) sorban egymás után szeretnénk kinyerni.
- Egy ilyen gyűjteményre olyan sorozatként gondolunk (a reprezentációja többnyire tényleg egy sorozat), amelyre egyetlen műveletet engedünk csak meg: a sorozat első elemének „kiolvasását” (leszakítását).
- Az olvasást az st, e, x : read szimbólummal jelöljük (ami egy értékadásra utal: st, e, x:=read(x)), ahol x:infile(E), e:E, st:Status = {abnorm, norm}, és amely az alábbi tevékenységet fedi:

if x = <> **then** st := abnorm

else st, e, x := norm, x_1, <x_2 , … , $x_{|x|}$>

Szekvenciális outputfájl

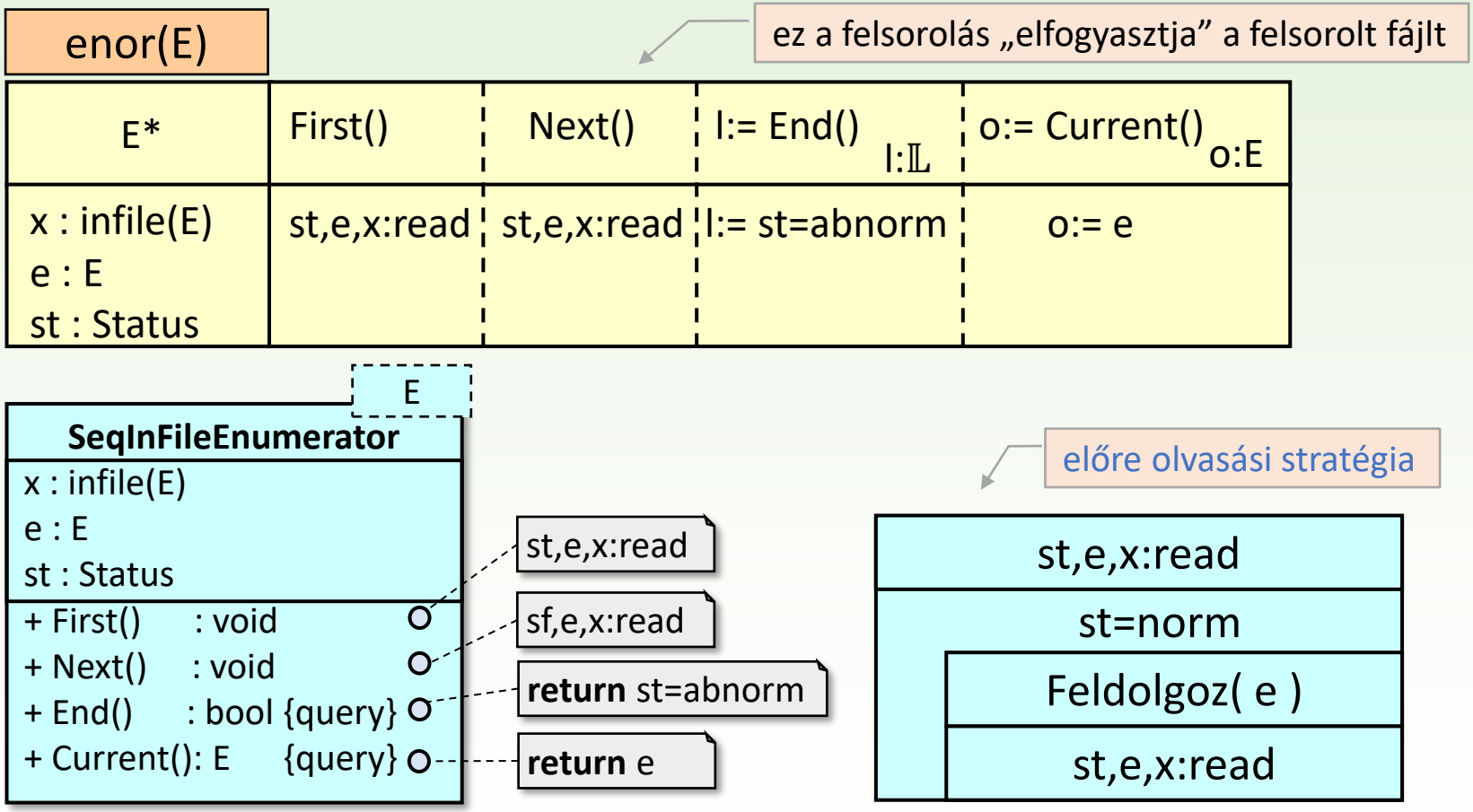
- A szekvenciális outputfájl (outfile(E)) elnevezést olyan gyűjteményre használjuk, amelybe sorban egymás után helyezhetünk el egy adott E halmazhoz tartozó elemeket.
- Egy ilyen gyűjteményre olyan sorozatként gondolunk (a reprezentációja többnyire tényleg egy sorozat), amelyre két műveletet vezetünk be:
  1. kezdetben az üres sorozat létrehozását
  2. a sorozat végéhez történő hozzáírást (hozzáfűzést)
- Az inicializálást az x := <> értékadás végzi, ahol x:outfile(E), és <> az üres sorozat.
- Az írást az x : write(e) szimbólummal jelöljük (ami az x := write(x, e) értékadásra utal), ahol x:outfile(E), e:E, és amely az alábbi tevékenységet fedi: x := x $\oplus$ <e>.

Fájlfeldolgozásos feladatok

- A gyakorlatban sokszor találkozhatunk olyan feladatokkal, amelyekben szekvenciális inputfájlból (esetleg több fájlból) állítunk elő szekvenciális outputfájlt (esetleg fájlokat) előállítani:
  - másolás
  - kiválogatás
  - szétválogatás
  - összefuttatás
- Ezeknél a feladatoknál a szekvenciális inputfájlt elemenként dolgozzuk fel úgy, hogy az inputfájl felsorolásához annak olvasó műveletét használjuk, a megoldást pedig az összegzés algoritmus mintára vezetjük vissza.

Szekvenciális inputfájl felsorolója

E-beli értékeket tartalmazó szekvenciális inputfájl elemeinek felsorolása



Összegzés fájlkezeléshez

Általános összegzés

A = (t:enor(E), s:H)

Ef = ($t = t_0$)

Uf = ($s = \Sigma_{e \in t_0} f(e)$)

$f : E \to H$
$+ : H \times H \to H$
$0 \in H$ neutr. elem

Fájlfeldolgozás

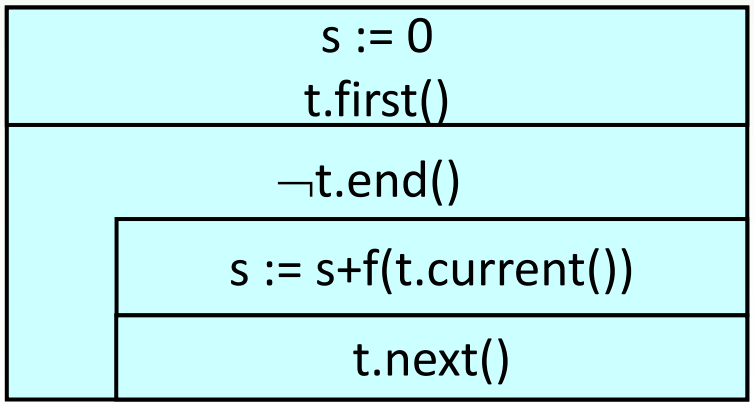
A = (x:infile(E), y:outfile(F))

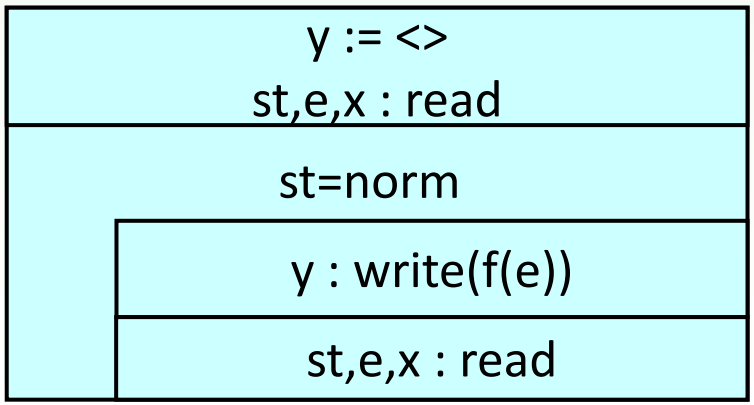
Ef = ($x = x_0$)

Uf = ($y = \bigoplus_{e \in x_0} f(e)$)

$f : E \to F^*$
$\bigoplus : F^* \times F^* \to F^*$
$\langle\rangle \in F^*$ neutr. elem

Összegzés:
t:enor(E) ~ x:infile(E) felsorolása a st,e,x : read művelettel
H,+,0 ~ F*, ⊕, <>





C++ nyelvi elemek

- Szöveges állomány hátterű szekvenciális inputfájlból ifstream típusú adatfolyam-objektummal olvashatjuk az adatokat (#include <fstream>).
- Az st,e,x:read műveletet olyan utasítások helyettesítik, amelyek igaz értéket adnak vissza, ha st=norm.
  Ezen az utasítások az e típusának függvényei.
  - karakter: x.get(e)
    ez minden karaktert (elválasztó jeleket is) beolvas
  - definiált lexikális egység: x >> e
    kikapcsolható az elválasztó jeleket (white space) átlépő mechanizmus: x.unsetf(ios::skipws)

- Szöveges állomány hátterű szekvenciális outputfájlba ofstream típusú adatfolyam-objektummal írhatunk (#include <fstream>).
- Az y:write(e) műveletet általában az y << e utasítás helyettesíti, de karakterek kiírására használható az y.put(e) is.

1.Feladat

Alakítsunk át egy ékezeteket tartalmazó szöveget (szöveges állományt) ékezet nélkülire (az eredmény egy másik szöveges állományba kerüljön)!

A = (x:infile(Char) , y:outfile(Char))

Ef = ($x = x_0$)

Uf = (y = $\bigoplus_{ch \in x_0}$ <átalakít(ch)>)

ahol átalakít : Char → Char és átalakít(ch) = …

másolás-átalakítás

Összegzés:

t:enor(E)	~	x:infile(Char) st,ch,x : read
f(e)	~	<átalakít(ch)>
H,+,0	~	Char*, ⊕, <>

y := < >
st, ch, x : read

st = norm

y : write(átalakít(ch))

st, ch, x : read

Szürkedoboz tesztelés

- Az összegzés teszteléséhez vizsgálni kell
  - a felsorolót
    - felsorolás hossza szerint: 0, 1, 2, illetve több elem felsorolása
    - felsorolás eleje, vége szerint: két eltérő betű felsorolásával már ellenőrizhető
  - a terheléses teszt most nem túl érdekes, hiszen az outputfájlt hossza meg fog egyezni az inputfájléval
- Ezeken kívül ellenőrizni kell a konverziót.

felsoroló szerint	hosszra:	x = <>, <a>, <ab>, <…>	→	y = <>, <a>, <ab>, <…>
	elejére:	x = <ab>, <áé>	→	y = <ab>, <ae>
	végére:	x = <ab>, <áé>	→	y = <ab>, <ae>
átalakítás szerint	ék-es mgh:	x = <ááíöőúüű>	→	y = <aeioouuu>
	ék-tlen mgh:	x = <aeioouuu>	→	y = <aeioouuu>
	msh:	x = <bsmnz>	→	y = <bsmnz>

C++ program

```
int main()
{
	ifstream x( "input.txt" );
	if ( x.fail() ){ cout << "Wrong file name!\n"; return 1;}
	ofstream y( "output.txt" );
	if ( y.fail() ){ cout << "Wrong file name!\n"; return 1;}

	char ch;
	while(x.get(ch)){
		y << transform(ch);
	}

	return 0;
}
```

sikeres olvasás esetén igaz értéket ad vissza

```
x.get(ch);
while(!x.eof()){
	y << transform(ch);
	x.get(ch);
}
```

Karakterek átalakítása

```
char transform(char ch)
{
   char new_ch;
   switch (ch) {
      case 'á'   :                              new_ch = 'a'; break;
      case 'é'    :                             new_ch = 'e'; break;
      case 'í'    :                             new_ch = 'i’;  break;
      case 'ó'   : case 'ö' : case 'õ' :        new_ch = 'o'; break;
      case 'ú'   : case 'ü' : case 'û' :        new_ch = 'u'; break;
      case 'Á'   :                              new_ch = 'A'; break;
      case 'É'   :                              new_ch = 'E'; break;
      case 'Í'   :                              new_ch = 'I’;  break;
      case 'Ó'   : case 'Ö' : case 'Õ':         new_ch = 'O'; break;
      case 'Ú'   : case 'Ü' : case 'Û':         new_ch = 'U'; break;
      default    :                              new_ch = ch;
   }
   return new_ch;
}
```

2.Feladat

Egy sportoló adott időközönként feljegyezte a pulzusszámát, és ezeket egy szekvenciális inputfájlban időpont-pulzusszám (sztring-természetes szám) párok sorozataként rögzítette. A fájl időpont szerint növekedően rendezett. Írjuk ki a konzolablakba a legalább 100-as pulzusszámú méréseket.

A = (x:infile(Mérés) , cout:outfile(Mérés))
Mérés=rec(idő:$\mathbb{S}$, pulzus:$\mathbb{N}$)

Ef = ($x = x_0 \wedge x \nearrow_{\text{idő}}$)

Uf = (cout = $\bigoplus_{e \in x_0, e.\text{pulzus} \geq 100} \langle e \rangle$)

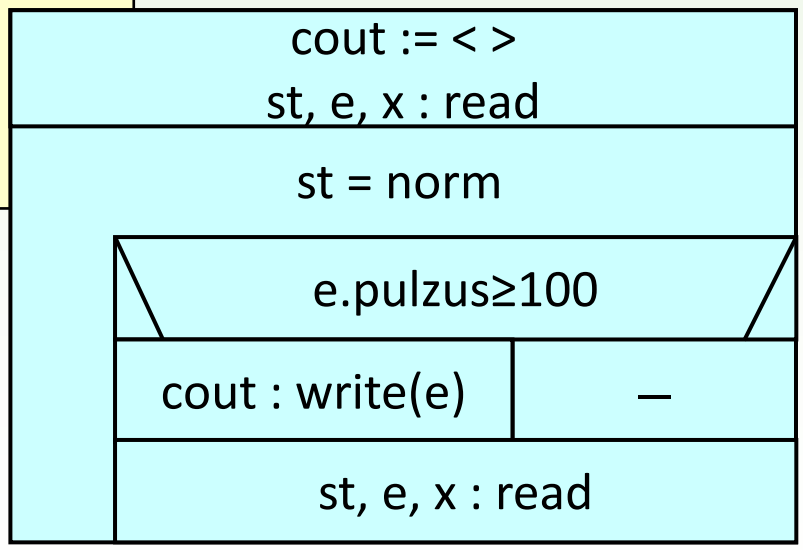
kiválogatás

Összegzés:

t:enor(E) ~ x:infile(Mérés)
st,e,x : read

f(e) ~ <e> ha e.pulzus≥100

H,+,0 ~ Mérés*, ⊕, <>



Szürkedoboz tesztelés

- Vizsgálni kell
  - a felsorolót
    - felsorolás hossza: 0, 1, 2, több hosszú csupa 100 feletti mérés
    - felsorolás eleje: 100 feletti ill. 100 alatti mérés elől
    - felsorolás vége: 100 feletti ill. 100 alatti mérés hátul
  - a terheléses teszt most sem érdekes
  - a kiválogatás feltétele (kisebb, nagyobb, egyenlő)

C++ program

```
struct Data{ string time; int pulse; };

int main()
{
	ifstream x;
	bool error = true;
	do{
		string fname;
		cout << "file name: "; cin >> fname;
		x.open(fname.c_str());
		if( (error=x.fail()) ) { cout << "Wrong file name!\n"; x.clear(); }
	}while(error);

	cout << "Selected measurements: ";
	Data e;
	while(x >> e.time >> e.pulse) {
		if(e.pulse>=100)
			cout << "ido: " << e.time << " ,pulzus: " << e.pulse << endl;
	}

	return 0;
}
```

```
x >> e.time >> e.pulse;
while(!x.fail()){
	if(e.pulse>=100) { cout << ... }
	x >> e.time >> e.pulse;
}
```

a read egy dupla olvasás

3.Feladat

Egy sportoló adott időközönként feljegyezte a pulzusszámát, és ezeket egy szekvenciális inputfájlban időpont-pulzusszám (sztring-természetes szám) párok sorozataként rögzítette. A fájl időpont szerint növekedően rendezett. Hányszor mért 100-nál alacsonyabb pulzusszámot azt megelőzően, hogy először elérte a pulzusa a legalább 100-at, és mekkora volt a legalacsonyabb pulzusszáma ezt követően? Legalább egyszer biztosan mért 100-as pulzusszámot, és azt követően is voltak még mérések.

A felsorolás nem a végéig tart

A = (x:infile(Mérés) , c:$\mathbb{N}$, min:$\mathbb{N}$) Mérés=rec(idő:$\mathbb{S}$, pulzus:$\mathbb{N}$)

Ef = ($x = x_0 \wedge x \nearrow_{\text{idő}} \wedge \exists i \in [1..|x|-1] : x_i.\text{pulzus} \geq 100$)

a minimum kiválasztás másik eredménye most nem érdekes

Uf = ((c, (st',e',x')) = $\sum_{e \in x_0}^{e.\text{pulzus}<100} 1 \wedge (\text{min}, .) = \text{MIN}_{e \in x'}(e.\text{pulzus})$)

st, e, x változók értéke a feldolgozás ezen szakasza után (e'≥100)

folytatja x felsorolását

Összegzés:

t:enor(E) ~ x:infile(Mérés), st,e,x:read
amíg: e.pulzus<100

f(e) ~ 1

H, +, 0 ~ $\mathbb{N}$, +, 0

Maximum kiválasztás:

t:enor(E) ~ x:infile(Mérés), st,e,x:read
felsorolás folytatása

f(e) ~ e.pulzus

H, > ~ $\mathbb{N}$, <

Algoritmus

Összegzés:

t:enor(E) ~ x:infile(Mérés), st,e,x:read
amíg: e.pulzus<100

f(e) ~ 1

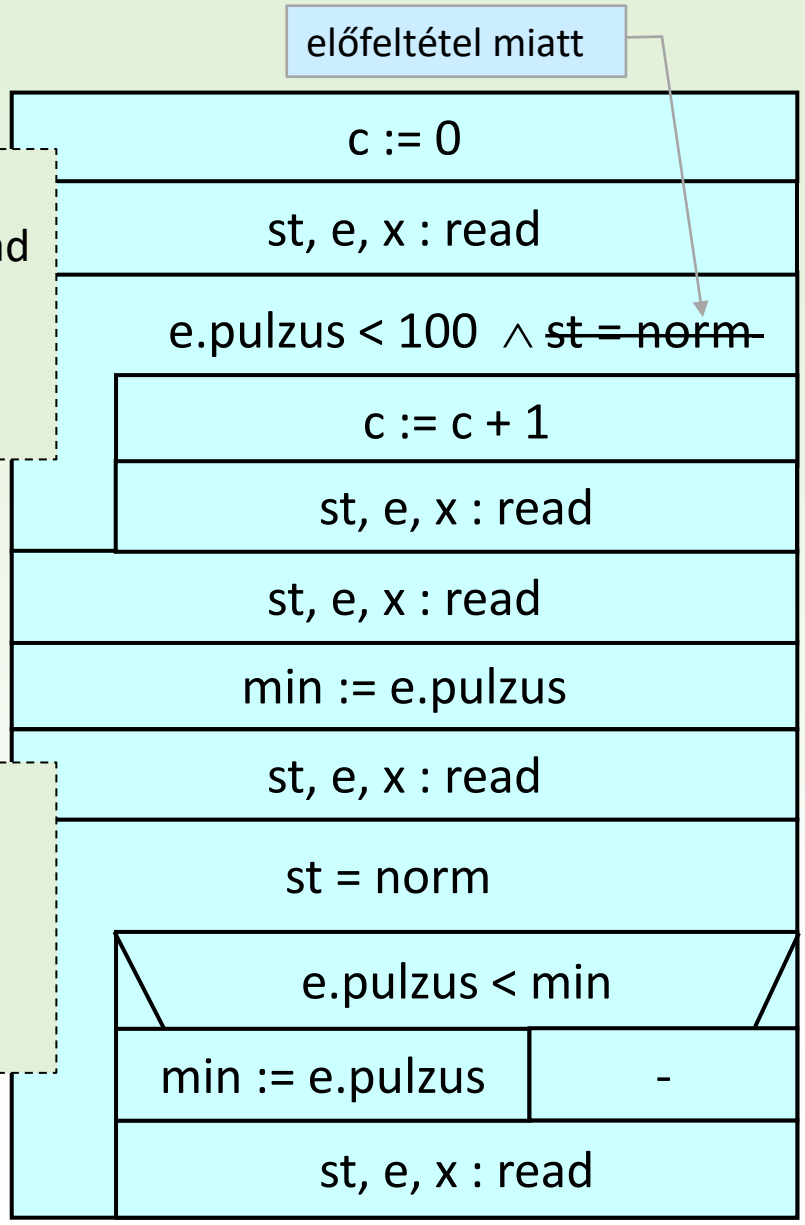
H, +, 0 ~ $\mathbb{N}$, +, 0

Maximum kiválasztás:

t:enor(E) ~ x:infile(Mérés), st,e,x:read
felsorolás folytatása

f(e) ~ e.pulzus

H, > ~ $\mathbb{N}$, <



Kitérő

Hogyan módosul a megoldás, ha a minimum kiválasztásnak figyelembe kellene vennie az első legalább 100-as pulzusszámot is.

A = (x:infile(Mérés) , c:$\mathbb{N}$, min:$\mathbb{N}$) Mérés=rec(idő:$\mathbb{S}$, pulzus:$\mathbb{N}$)

Ef = ($x = x_0 \wedge x \nearrow_{\text{idő}} \wedge \exists i \in [1..|x|] : x_i.\text{pulzus} \geq 100$)

Uf = ((c, (st',e',x')) = $\sum_{e \in x_0}^{e.\text{pulzus}<100} 1 \wedge$ (min, .) = $\text{MIN}_{e \in (e', x')}$(e.pulzus))

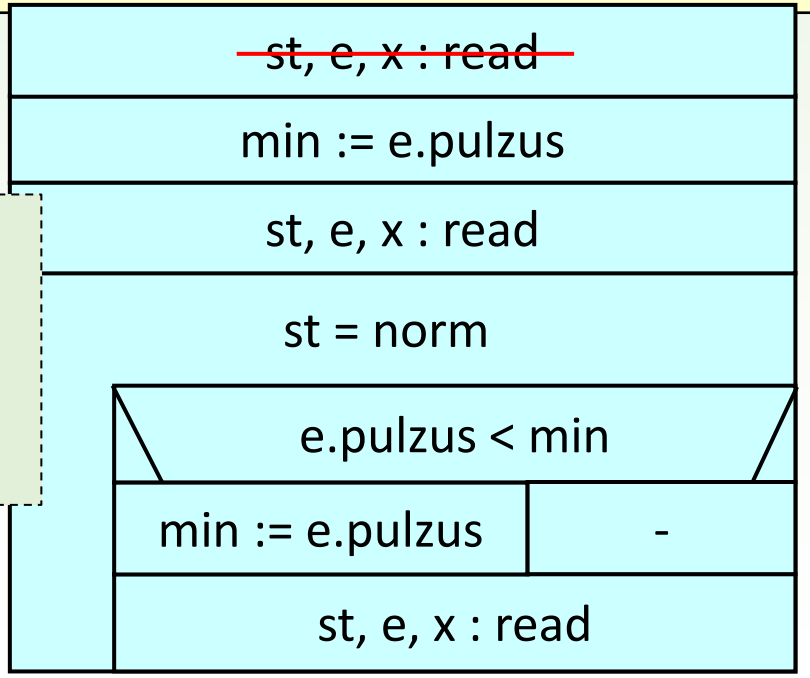
figyelembe vesszük a már kiolvasott e' értéket is

Maximum kiválasztás:

t:enor(E) ~ x:infile(Mérés), st,e,x:read
első olvasás nélkül

f(e) ~ e.pulzus

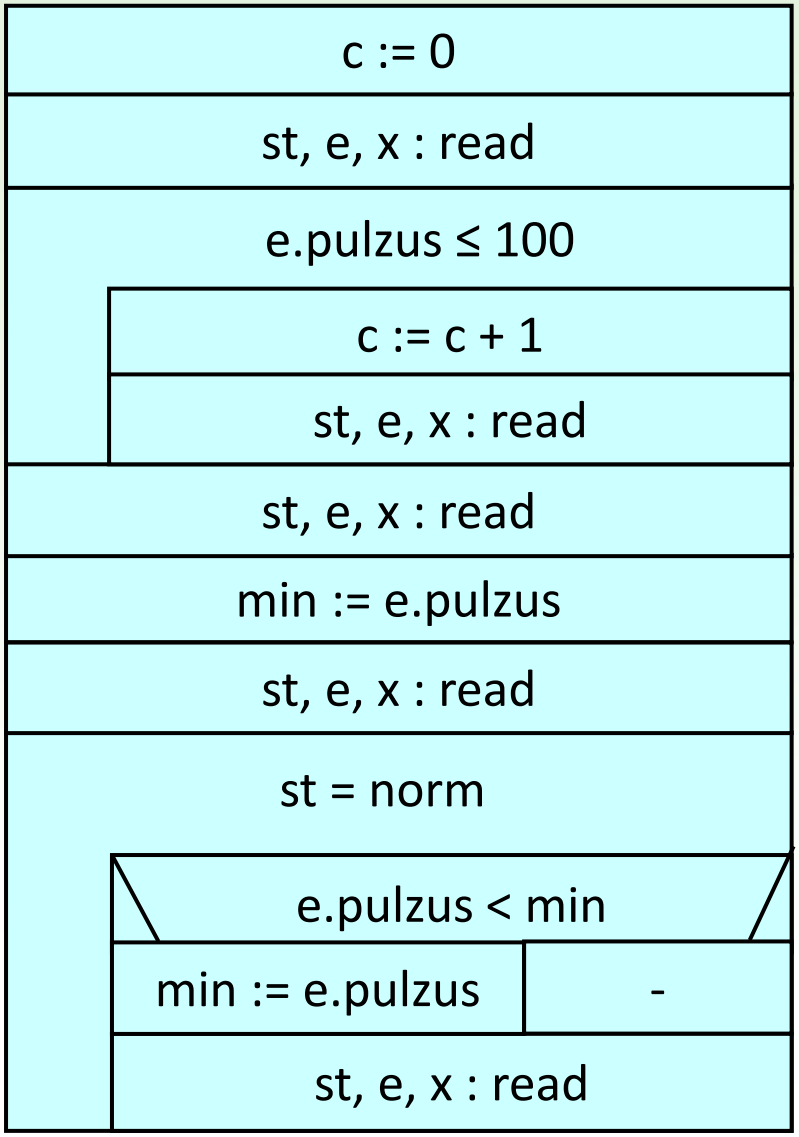
H, > ~ $\mathbb{N}$, <



Tesztelés

- Tesztelni kell külön-külön a felsoroló első szakaszán működő összegzést, és a hátsó szakaszán működő minimum kiválasztást.
  - Olyan inputfájlok kellenek, ahol a fájl első szakasza lehet 0, 1, 2 hosszú, vagy hosszabb, hátsó szakasza 1, 2 hosszú, vagy hosszabb.
  - Vizsgálni kell, hogy mindkét szakasz mindkét végét figyelembe veszi-e a számítás: az összegzésnél ehhez elég ha a fájl első két mérése 100-nál kisebb pulzusszámú, a minimum kiválasztásnál kell olyan input, amikor a hátsó szakasz első eleme tartalmazza a legkisebb pulzus számot, és olyan is, amikor az utolsó eleme.
  - A minimum kiválasztás tesztelésénél vizsgálni kell azt is, amikor a hátsó szakasz közepén van a legkisebb pulzus számú elem, illetve amikor több egyformán legkisebb pulzus számú elem is van.
- Végül érdemes az e.pulzus<100 feltételt is tesztelni 99, 100, 101 pulzus számokkal.

Kódolás




```
c = 0;
x >> e.time >> e.pulse;
while(e.pulse<100){
      ++c;
      x >> e.time >> e.pulse;
}
```

```
x >> e.time >> min;
x >> e.time >> e.pulse;
while( !x.fail() ){
      if (e.pulse<min) min = e.pulse;
      x >> e.time >> e.pulse;
}
```

C++ program

```
struct Data{
	string time;
	int pulse;
};

int main()
{
	ifstream x("input.txt");
	Data e;

	int c = 0;
	while(x >> e.time >> e.pulse && e.pulse<100) {
		++c ;
	}
	int min;
	x >> e.time >> min;
	while(x >> e.time >> e.pulse) {
		if (e.pulse<min) min = e.pulse;
	}
	cout << "count: " << c << endl  << "min: " << min << endl;
	return 0;
}
```

```
c = 0;
x >> e.time >> e.pulse;
while(e.pulse<100){
	++c;
	x >> e.time >> e.pulse;
}
```

```
x >> e.time >> min;
x >> e.time >> e.pulse;
while( !x.fail() ){
	if (e.pulse<min) min = e.pulse;
	x >> e.time >> e.pulse;
}
```

4.Feladat

Több sportolónak mértük edzés közben a pulzusát, és a mérési adatokat soronként rögzítettük egy szöveges állományban.

Minden sor egy sportoló (több sztringből álló) nevével kezdődik, amelyet a mérési adatok követnek. Egy mérési adat egy időpontból (mm:ss formájú sztring), és egy pulzusszámból (természetes szám) áll. A sor adatait elválasztó jelek (szóközök, tabulátorjelek) választják el egymástól. Feltehetjük, hogy a mérések időpont szerint növekedően rendezettek.

Gyűjtsük ki azokat a sportolókat, akiknek pulzusszáma az első legalább 100-as mérés után 100 alá esett vissza.

```
Gipsz Jakab Elemér              00:01 67 01:15 89   02:55 102 04:03 108
Szer Elek                       00:01 72 00:55 102 03:15 110
Jose Fernando Llano del Colona  00:01 67 02:30 100 04:55 95   06:03 105
```

Elemzés

fizikai elhelyezkedést szem előtt tartó

A = (x : infile(Sor) , y : outfile(Sor)) Sor = $\mathbb{S}$*,
Ef = (x = $x_0 \wedge \forall e \in x$: e olyan, hogy …)
Uf = (…)

bemenetet szem előtt tartó

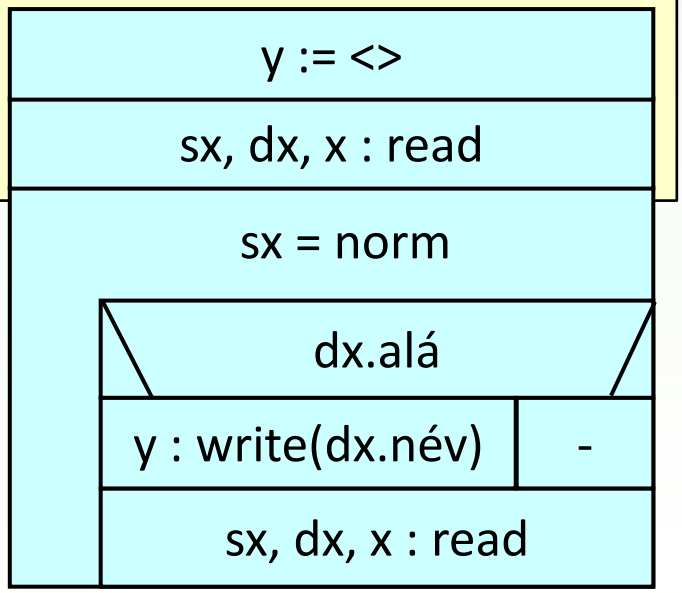
A = (x : infile(Sportoló) , y : outfile($\mathbb{S}$))
Sportoló = rec(név:$\mathbb{S}$, mérések:Mérés*) Mérés=rec(idő:$\mathbb{S}$, pulzus:$\mathbb{N}$)
Ef = (x = $x_0 \wedge \forall e \in x$: e.mérések $\nearrow_{\text{idő}}$)
Uf = (…)

kimenetet szem előtt tartó

A = (x : infile(Sportoló) , y : outfile($\mathbb{S}$))
Sportoló = rec(név:$\mathbb{S}$, alá:$\mathbb{L}$)
Ef = (x = x_0)
Uf = (y = $\bigoplus_{dx \in x_0}$ <dx.név>)
dx.alá

Összegzés (kiválogatás):

t:enor(E)	~	x:infile(Sportoló)
e∈t	~	sx,dx,x :read
f(e)	~	ha dx.alá akkor <dx.név>
s	~	y
H,+,0	~	$\mathbb{S}$*, $\oplus$, <>

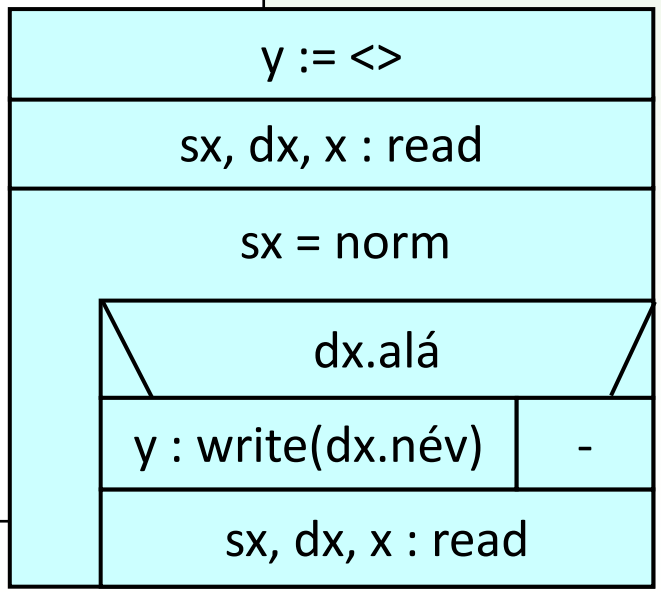


Főprogram

Az áttekinthetőséget, valamint a megoldás absztrakt és konkrét elemeinek szétválasztását segíti a saját típusok bevezetése: InpFile, Athlete, Status

```
int main()
{
	try{
		InpFile x("input.txt");
		Athlete dx;
		Status sx;

		while( x.read(dx,sx) ) {
			if (dx.below) {
				cout << dx.name << endl;
			}
		}
	} catch( InpFile::FileOpenErrorException) {
		cout  << "Wrong file name!\n";
	}
	return 0;
}
```



Felhasználói típusok

Az áttekinthetőséget, valamint a megoldás absztrakt és konkrét elemeinek szétválasztását segíti a saját típusok bevezetése: InpFile, Athlete, Status

```
struct Athlete {
        std::string name;
        bool below;
};
enum Status {abnorm, norm};

class InpFile{
public:
        class FileOpenErrorException : std::except

        InpFile(std::string fname){
                f.open(fname.c_str());
                if(f.fail()) throw new FileOpenErrorException;
        }
    bool read(Athlete &dx, Status &sx);
private:
    std::ifstream f;
};
```

Sportoló = rec(név:𝕊, alá:𝕃)

Státusz = { abnorm, norm }

infile(Sportoló)

Sportoló*	sx,dx,x:read
f : ifstream	lásd később

sorok sorozata

Az olvasó (read) művelet

Ennek a műveletnek egy újabb sportoló adatait kell előállítani, és ehhez az eredetileg sorokat tartalmazó szekvenciális inputfájlra kell támaszkodnia. Ezért egy soronkénti olvasásra van szükség.

```
bool InpFile::read(Athlete &dx, Status &sx)
{
      string line;
      getline(f, line);
      if (!f.fail() && line!="")
      {
            sx=norm;
            ... // feldolgozás
      }
      else sx=abnorm;
      return norm==sx;
}
```

infile(Sportoló)	
Sportoló*	sx,dx,x:read
f : infile(Sor)	lásd lenn

ifstream

A = (f : infile(Sor), dx : Sportoló, sx : Status)
Sportoló = rec(név:$\mathbb{S}$, alá:$\mathbb{L}$)

Ef = (f = f')

Uf = ((| f' |> 0 ⟶ sx = norm ∧ dx = feldolgoz(f'$_1$) ∧ f = < f'$_2$, … , f'$_{|f'|}$>) ∧ (| f' |= 0 ⟶ sx = abnorm))

az aktuális sor

amikor nincs több sora az inputfájlnak

Egy sor feldolgozása

Egy szavakra (tokenekre) bontott sorban az első néhány szó alkotja a sportoló nevét, majd az első időpont adattól kezdve jönnek a mérések (időpont-pulzusszám párok). Döntsük el van-e legalább 100-as pulzus számú és azt követően 100-nál kisebb pulzus számú mérés.

Gipsz Jakab Elemér 00:01 67 01:15 89 02:55 102 04:03 108

a sort szavakra bontjuk

$A = (t : \mathbb{S}^*, dx : \text{Sportoló})$ $\quad \text{Sportoló} = \text{rec}(\text{név}:\mathbb{S}, \text{alá}:\mathbb{L})$

$Ef = (t = t')$

összefűzi a sor eleji szavakat, amíg nem talál dátumot.

az első dátum

$Uf = (\quad (dx.\text{név}, (\text{szó}', t'')) = \bigoplus_{\text{szó}\in t'}^{\text{szó}[3]\neq':'} <\text{szó}> \ \wedge$

kiválasztja az első legalább 100-as pulzusszámú mérést

még nem vizsgált adatok

$\wedge \quad l1, ((\text{idő}', \text{pulzus}') , t''') = \text{SEARCH}_{(\text{idő},\text{pulzus})\in(\text{szó}',t'')} (\text{pulzus}\geq 100) \ \wedge$

még nem vizsgált adatok

felsorolás folytatása úgy, hogy innentől méréseket kell olvasni

eldönti van-e kisebb mérés 100-as pulzusszámnál

$\wedge \quad l2 = \text{SEARCH}_{(\text{idő},\text{pulzus})\in((\text{idő}',\text{pulzus}'),t''')} (\text{pulzus}<100)$

mérések felsorolásának folytatása előre olvasás nélkül

$\wedge \quad dx.\text{alá} = (l1 \wedge l2) \quad)$

Egy sor feldolgozása

egy sor adatainak olvasásához
#include <sstream>

```
...
istringstream in(line);
string token;
dx.name = "";
while( in >> token && token[2]!=':' ) {
      dx.name += token + " ";
}
string time = token;
int pulse;
in >> pulse;
bool l1 = false;
while( !l1 && !in.fail() ) {
      l1 = pulse>=100;
      in >> time >> pulse;
}
bool l2 = false;
while( !dx. below && !in.fail() ) {
      l2 = pulse<100;
      in >> time >> pulse;
}
dx.below = l1 && l2;
...
```

névtagok olvasása

összefűzés feltétel fennállásáig

lineáris keresés

idő-pulzusszám párok olvasása

lineáris keresés

Szürkedoboz tesztelés vázlata

kiválogatás (feltételes összegzés):

a felsoroló hossza szerint:	0, 1, 2, több feltételnek megfelelő sportoló
a felsorolás eleje/vége:	csak az első és utolsó sportoló felel meg a feltételnek
terhelés:	nem kell

névtagok összefűzése (összegzés):

a felsoroló hossza szerint:	1, 2, több névtagú sportoló
a felsorolás eleje/vége:	előzővel letudva

legalább 100-as mérés keresése (lineáris keresés) :

a felsoroló hossza szerint:	0, 1, 2, több mérés az első legalább 100-as mérés előtt
a felsorolás eleje/vége:	előzővel letudva
eredmény szerint:	99, 100, 101

100 alatti mérés keresése (lineáris keresés):

a felsoroló hossza szerint:	0, 1, 2, több mérés az első legalább 100-as mérés után
a felsorolás eleje/vége:	elején, illetve a végén 100 alatti mérés
eredmény szerint:	99, 100, 101